HW5 Coding Part

In [1]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
from torch.utils.data import TensorDataset, DataLoader, random_split
import torchvision.transforms as transforms
import sklearn
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from tqdm import tqdm
from sklearn.metrics import classification_report
```

Problem 0: Pytorch Tutorial (12 points)

Tensors

Tensors can be created from numpy data or by using pytorch directly.

In [2]:
```
x_data = [[1, 2], [3, 4]]
x = torch.tensor(x_data)

np_array = np.array(x_data)
x_np = torch.from_numpy(np_array)

shape = (2,3)
rand_tensor = torch.rand(shape)
np_rand_array = rand_tensor.numpy()

print(f"Tensor from np: \n {x_np} \n")
print(f"Rand Tensor: \n {rand_tensor} \n")
print(f"Rand Numpy Array: \n {np_rand_array} \n")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {device} \n")
```

```
Tensor from np:
 tensor([[1, 2],
        [3, 4]])

Rand Tensor:
 tensor([[0.8240, 0.4417, 0.2339],
        [0.1085, 0.7467, 0.8165]])

Rand Numpy Array:
 [[0.82399017 0.4417374  0.23394042]
 [0.10847783 0.74671054 0.816483  ]]

Device: cuda
```

1) Tensor squeezing, unsqueezing and viewing (3 points)

Tensor squeezing, unsqueezing and viewing are important methods to change the dimension of a Tensor, and the corresponding functions are torch.squeeze, torch.unsqueeze and torch.Tensor.view. Please read the documents of the functions, and finish the following practice.

In [3]:
```
# x is a tensor with size being (3, 2)
x = torch.Tensor([[1, 2],[3, 4],[5, 6]])
print("Original shape:", x.shape)  # Should be (3, 2)

############## YOUR CODE HERE ##############
# TODO: Add two new dimensions to x by using the function torch.unsqueeze
# Hint: Add dimension at position 1 (second dimension)
# Hint: Add dimension at position 3 (fourth dimension)
x = torch.unsqueeze(x, 1) # Add dimension at position 1
x = torch.unsqueeze(x, 3) # Add dimension at position 3

############## END YOUR CODE ##############
print(x.shape) # Should be torch.Size([3, 1, 2, 1])

############## YOUR CODE HERE ##############
# TODO: Remove the two dimensions just added by using the function torch.
# Hint: Remove the dimension at position 3
# Hint: Remove the dimension at position 1
x = torch.squeeze(x, 3) # Remove dimension at position 3
x = torch.squeeze(x, 1) # Remove dimension at position 1

############## END YOUR CODE ##############
print(x.shape) # Should be torch.Size([3, 2])

############## YOUR CODE HERE ##############
# TODO: x is now a two-dimensional tensor, or in other words a matrix. No
x = x.view(6)
############## END YOUR CODE ##############
print("After view:", x.shape)  # Should be torch.Size([6])
```

```
Original shape: torch.Size([3, 2])
torch.Size([3, 1, 2, 1])
torch.Size([3, 2])
After view: torch.Size([6])
```

2) Tensor concatenation and stack (2 points)

Tensor concatenation and stack are operations to combine small tensors into big tensors. The corresponding functions are torch.cat and torch.stack. Please read the documents of the functions, and finish the following practice.

Hints:

1. `torch.stack((obj1, obj2))` : A new dimension is automatically added (dim=0 by default) and the input tensors are then stacked along that dimension.
2. `torch.cat((obj1, obj2))` : Dimensions are not added, but are spliced directly over existing dimensions, you should consider this case.

In [4]:
```
# x is a tensor with size being (3, 2)
x = torch.Tensor([[1, 2], [3, 4], [5, 6]])

# y is a tensor with size being (3, 2)
y = torch.Tensor([[-1, -2], [-3, -4], [-5, -6]])

############## YOUR CODE HERE ##############
# TODO: Our goal is to generate a tensor z with size as (2, 3, 2), and z[
z = torch.stack((x, y), dim=0) # Stack x and y along the first dimension
############## END YOUR CODE ##############
print(z)

############## YOUR CODE HERE ##############
# TODO: Use torch.cat and torch.unsqueeze to generate z
z = torch.cat((x.unsqueeze(0), y.unsqueeze(0)), dim=0) # Concatenate x an
############## END YOUR CODE ##############
print(z)

# The tow outputs are expected to be tensor([[[ 1.,  2.], [ 3.,  4.], [ 5
```

```
tensor([[[ 1.,  2.],
         [ 3.,  4.],
         [ 5.,  6.]],

        [[-1., -2.],
         [-3., -4.],
         [-5., -6.]]])
tensor([[[ 1.,  2.],
         [ 3.,  4.],
         [ 5.,  6.]],

        [[-1., -2.],
         [-3., -4.],
         [-5., -6.]]])
```

3) Tensor expansion (2 points)

Tensor expansion is to expand a tensor into a larger tensor along singleton dimensions. The corresponding functions are torch.Tensor.expand and torch.Tensor.expand_as. Please read the documents of the functions, and finish the following practice.

In [5]:
```
# x is a tensor with size being (3)
x = torch.Tensor([1, 2, 3])

# Our goal is to generate a tensor z with size (2, 3), so that z[0,:,:] =

############## YOUR CODE HERE ##############
# TODO: Change the size of x into (1, 3) by using torch.unsqueeze.
x = torch.unsqueeze(x, 0) # Add dimension at position 0
############## END YOUR CODE ##############
print(x) # Output is expected to be tensor([[1., 2., 3.]])

############## YOUR CODE HERE ##############
# TODO: Then expand the new tensor to the target tensor by using torch.Te
z = x.expand(2, 3) # Expand x to size (2, 3)
############## END YOUR CODE ##############
print(z) # Output is expected to be tensor([[1., 2., 3.], [1., 2., 3.]])
```

```
tensor([[1., 2., 3.]])
tensor([[1., 2., 3.],
        [1., 2., 3.]])
```

4) Tensor reduction in a given dimension (5 points)

In deep learning, we often need to compute the mean/sum/max/min value in a given dimension of a tensor. Please read the document of torch.mean, torch.sum, torch.max, torch.min, torch.topk, and finish the following practice.

In [6]:
```
# x is a random tensor with size being (10, 50)
x = torch.randn(10, 50)

############## YOUR CODE HERE ##############
# TODO: Compute the mean value for each row of x.
# You need to generate a tensor x_mean of size (10), and x_mean[k, :] is
# dim = 1: eliminate the second(1)'s dimension
x_mean = torch.mean(x, dim=1)
############## END YOUR CODE ##############
print(x_mean.shape)

############## YOUR CODE HERE ##############
# TODO: Compute the sum value for each row of x.
# You need to generate a tensor x_sum of size (10).
x_sum = torch.sum(x, dim=1)
############## END YOUR CODE ##############
print(x_sum.shape)

############## YOUR CODE HERE ##############
# TODO: Compute the max value for each row of x.
# You need to generate a tensor x_max of size (10).
# Hint: torch.max() -> (max_val, indices)
(x_max, indices) = torch.max(x, dim=1)
############## END YOUR CODE ##############
print(x_max.shape)

############## YOUR CODE HERE ##############
# TODO: Compute the min value for each row of x.
# You need to generate a tensor x_min of size (10).
# Hint: torch.max() -> (min_val, indices)
(x_min, indices) = torch.min(x, dim=1)
```

```
############## END YOUR CODE ##############
print(x_min.shape)

############## YOUR CODE HERE ##############
# TODO: Compute the top-5 values for each row of x.
# You need to generate a tensor x_min of size (10. 5).
# Hint: torch.max() -> (min_val, indices)
(x_xtop, indices) = torch.topk(x, 5, dim=1)
############## END YOUR CODE ##############
print((x_xtop.shape))
```

```
torch.Size([10])
torch.Size([10])
torch.Size([10])
torch.Size([10])
torch.Size([10, 5])
```

Autograd (0 point) (Highly recommend checking it out)

This small section shows you how pytorch computes gradients. When we create tenors, we can set `requires_grad` to be true to indicate that we are using gradients. For most of the work that you actually do, you will use the `nn` package, which automatically sets all parameter tensors to have `requires_grad=True`.

In [7]:
```
# Below is an example of computing the gradient for a single data point i

x = torch.ones(5)  # input tensor
y = torch.zeros(1)  # label
# requires_grad = True : we are using this parameter's gradient
# use nn package, set all parameter tensors to have required_grad = True
w = torch.randn(5, 1, requires_grad=True)
b = torch.randn(1, requires_grad=True)
pred = torch.sigmoid(torch.matmul(x, w) + b)
loss = torch.nn.functional.binary_cross_entropy(pred, y)
loss.backward()  # Computers gradients
print("W gradient:", w.grad)
print("b gradient:", b.grad)

# when we want to actually take an update step, we can use optimizers:
optimizer = torch.optim.SGD([w, b], lr=0.1)  # [w,b] is the model.paramet
print("Weight before", w)
optimizer.step()  # use the computed gradients to update
# Print updated weights
print("Updated weight", w)

# Performing operations with gradients enabled is slow...
# You can disable gradient computation using the following enclosure:
with torch.no_grad():
    # Perform operations without gradients
    ...
```

```
W gradient: tensor([[0.3570],
        [0.3570],
        [0.3570],
        [0.3570],
        [0.3570]])
b gradient: tensor([0.3570])
Weight before tensor([[-0.4210],
        [ 0.5155],
        [-1.3975],
        [-0.1170],
        [ 0.3922]], requires_grad=True)
Updated weight tensor([[-0.4567],
        [ 0.4798],
        [-1.4332],
        [-0.1527],
        [ 0.3565]], requires_grad=True)
```

Devices (0 point) (Highly recommend checking it out)

Pytorch supports accelerating computation using GPUs which are available on google colab. To use a GPU on google colab, go to runtime -> change runtime type -> select GPU.

Note that there is some level of strategy for knowing when to use which runtime type. Colab will kick users off of GPU for a certain period of time if you use it too much. Thus, its best to run simple models and prototype to get everything working on CPU, then switch the instance type over to GPU for training runs and parameter tuning.

Its best practice to make sure your code works on any device (GPU or CPU) for pytorch, but note that numpy operations can only run on the CPU. Here is a standard flow for using GPU acceleration:

In [8]:
```
# Determine the device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device", device)
# Next create your tensors
tensor = torch.zeros(4, 4, requires_grad=True)
# Move the tensor to the device you want to use
tensor = tensor.to(device)

# Perform whatever operations you want.... (often this will involve gradi
# These operations will be accelerated by GPU.
tensor = 10*(tensor + 1)

# bring the tensor back to CPU, first detaching it from any gradient comp
tensor = tensor.detach().cpu()

# Convert to numpy if you want to perform numpy operations.
tensor_np = tensor.numpy()
```

```
Using device cuda
```

Build an NN (0 point) (Highly recommend checking it out)

Pytorch implements composable blocks in `Module` classes. All layers and modules in pytorch inherit from `nn.Module`. When you make a module you need to implement two functions: `__init__(self, *args, **kwargs)` and `foward(self, *args, **kwargs)`. Modules also have some nice helper functions, namely `parameters` which will recursively return all of the parameters. Here is an example of a logistic regression model:

In [9]:
```
class Perceptron(nn.Module):
  def __init__(self, in_dim):
    super().__init__()
    # This is a linear layer, it computes Xw + b
    self.layer = nn.Linear(in_dim, 1)

  def forward(self, x):
    return torch.sigmoid(self.layer(x)).squeeze(-1)

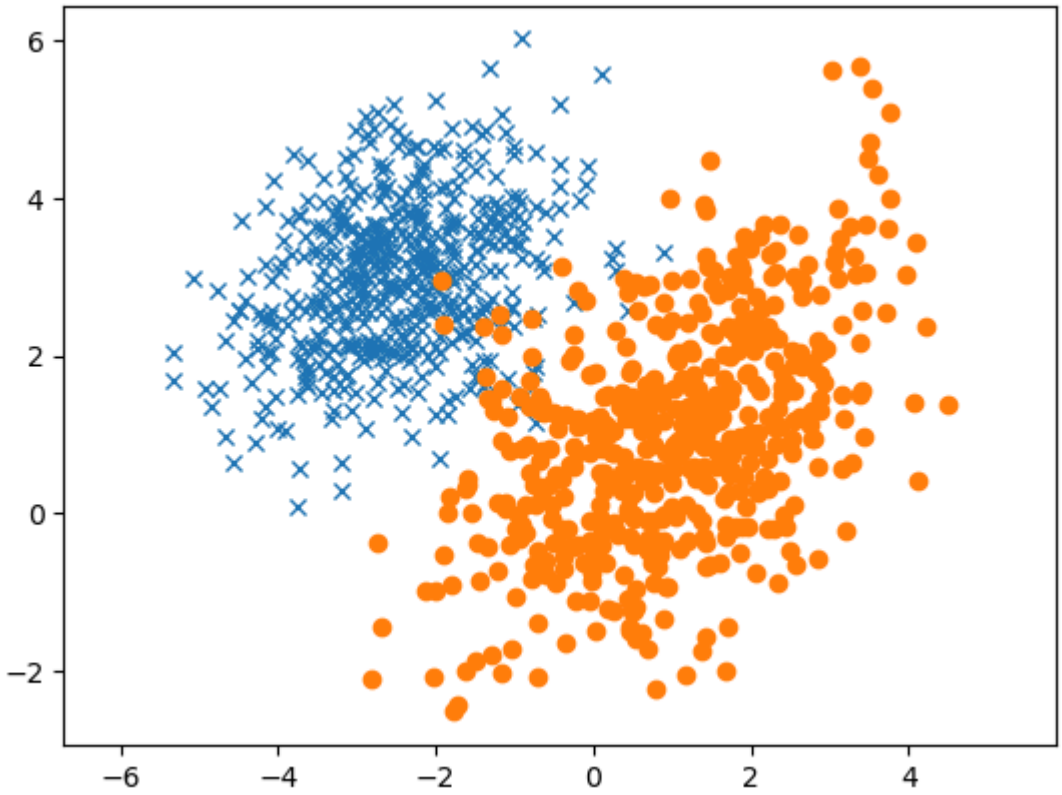

perceptron = Perceptron(10)
# Move all the perceptron's tensors to the device
perceptron = perceptron.to(device)
# module.parameters() return all the parameters in this module ; There W
print("Parameters", list(perceptron.parameters()))
```

```
Parameters [Parameter containing:
tensor([[ 0.1729, -0.0875,  0.0162,  0.1482,  0.1730, -0.0740,  0.0483,
0.0042,
          0.2425, -0.0515]], device='cuda:0', requires_grad=True), Paramet
er containing:
tensor([-0.0168], device='cuda:0', requires_grad=True)]
```

Datasets (0 point) (Highly recommend checking it out)

Pytorch has nice interfaces for using datasets. Suppose we create a logistic regression dataset as follows:

In [10]:
```
c1_x1, c1_x2 = np.random.multivariate_normal(
    [-2.5, 3], [[1, 0.3], [0.3, 1]], 500).T
c2_x1, c2_x2 = np.random.multivariate_normal([1, 1], [[2, 1], [1, 2]], 50
c1_X = np.vstack((c1_x1, c1_x2)).T
c2_X = np.vstack((c2_x1, c2_x2)).T
X = np.concatenate((c1_X, c2_X))
y = np.concatenate((np.zeros(500), np.ones(500)))
# Shuffle the data
permutation = np.random.permutation(X.shape[0])
X = X[permutation, :]
y = y[permutation]
# Plot the data
plt.plot(c1_x1, c1_x2, 'x')
plt.plot(c2_x1, c2_x2, 'o')
plt.axis('equal')
plt.show()
```



We can then create a pytorch dataset object as follows. Often times, the default pytorch datasets will create these objects for you. Then, we can apply dataloaders to iterate over the dataset in batches.

In [11]:
```
dataset = torch.utils.data.TensorDataset(
    torch.from_numpy(X), torch.from_numpy(y))
print(dataset)
# We can create a dataloader that iterates over the dataset in batches.
dataloader = torch.utils.data.DataLoader(dataset, batch_size=10, shuffle=
for x, y in dataloader:
    print("Batch x:", x)
    print("Batch y:", y)
    break

# Clean up the dataloader as we make a new one later, you can ignore it h
del dataloader
```

```
<torch.utils.data.dataset.TensorDataset object at 0x7fccd6dff7f0>
Batch x: tensor([[ 1.0829,  0.9213],
        [-1.7793, -2.5217],
        [-3.0675,  2.7116],
        [-2.2374,  1.5411],
        [-1.6641,  2.5395],
        [ 2.0350,  2.4607],
        [-2.6904,  2.0987],
        [ 0.8872,  1.1746],
        [-0.7498, -0.6478],
        [-2.2964,  4.1929]], dtype=torch.float64)
Batch y: tensor([1., 1., 0., 0., 0., 1., 0., 1., 1., 0.], dtype=torch.floa
t64)
```

Splitting Train, Validation and Test sets randomly.

In [12]:
```
#Training: 70%, Validation: 15%, Testing: 15%
train_size = int(0.7 * len(dataset))
val_size = int(0.15 * len(dataset))
test_size = len(dataset) - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(
    dataset, [train_size, val_size, test_size]
)

# Creat the data_loders
batch_size = 10
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=T
val_loader = DataLoader(val_dataset, shuffle=False)
test_loader = DataLoader(test_dataset, shuffle=False)

print(f"Training size: {len(train_dataset)}")
print(f"Validation size: {len(val_dataset)}")
print(f"Testing size: {len(test_dataset)}")
```

```
Training size: 700
Validation size: 150
Testing size: 150
```

Training Loop and Progress Bar (0 point) (Very important! Highly recommend checking it out)

Here is an example of training a full Logistic Regression model in pytorch. Note the extensive use of modules -- modules can be used for storing networks, computation steps etc.

In [13]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device", device)

epochs = 10
batch_size = 10
learning_rate = 0.01

num_features = dataset[0][0].shape[0]
model = Perceptron(num_features).to(device)
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
criterion = torch.nn.BCELoss()

model.train()  # Put model in training mode
for epoch in range(epochs):
    training_losses = []
    train_correct = 0
    ProgressBar = tqdm(train_loader)
    for x, y in ProgressBar:
        x, y = x.float().to(device), y.float().to(device)
        #  for every mini-batch during the training phase, we typically w
        # Remove the gradients from the previous step ;Sets the gradients
        optimizer.zero_grad()
        pred = model(x)   # The value predicted using our model
        # compute loss
        loss = criterion(pred, y)
        # Compute gradients.
        loss.backward()
        # update the parameter using the gradient computed
```

```
        optimizer.step()
        training_losses.append(loss.item())
        # In a classification task, the output of a neural network model
        # For example, in a 10-class classification task, the output of t
        # So for multi-class classification tasks, try to use: train_corr
        train_correct += torch.sum(torch.round(pred) == y).item()
    train_accuracy = train_correct / len(train_dataset)
    val_losses = []
    num_correct = 0
    with torch.no_grad():
        model.eval()  # Put model in eval mode
        for x_val, y_val in val_loader:
            x_val, y_val = x_val.float().to(device), y_val.float().to(dev
            pred_val = model(x_val)
            loss_val = criterion(pred_val, y_val)
            val_losses.append(loss_val.item())
            num_correct += torch.sum(torch.round(pred_val) == y_val).item
        model.train()  # Put model back in train mode
    print(f"Epoch {epoch+1}/{epochs} | "
          f"Train Loss: {np.mean(training_losses):.4f} | "
          f"Train Acc: {train_accuracy:.4f} | "
          f"Val Loss: {np.mean(val_losses):.4f} | "
          f"Val Acc: {num_correct / len(val_dataset):.4f}")

# We can run predictions on the data to determine the Testing accuracy.
model.eval()
test_correct = 0
with torch.no_grad():
    for x, y in test_loader:
        x, y = x.float().to(device), y.float().to(device)
        pred = model(x)
        test_correct += torch.sum(torch.round(pred) == y).item()

print(f"\nTest Accuracy: {test_correct / len(test_dataset):.2%}")
```

```
Using device cuda
100%|██████████| 70/70 [00:00<00:00, 403.14it/s]
Epoch 1/10 | Train Loss: 0.6418 | Train Acc: 0.6829 | Val Loss: 0.4209 | V
al Acc: 0.9000
100%|██████████| 70/70 [00:00<00:00, 1063.68it/s]
Epoch 2/10 | Train Loss: 0.3342 | Train Acc: 0.9043 | Val Loss: 0.3325 | V
al Acc: 0.9067
100%|██████████| 70/70 [00:00<00:00, 931.90it/s]
Epoch 3/10 | Train Loss: 0.2856 | Train Acc: 0.9057 | Val Loss: 0.3004 | V
al Acc: 0.9200
100%|██████████| 70/70 [00:00<00:00, 936.37it/s]
Epoch 4/10 | Train Loss: 0.2631 | Train Acc: 0.9057 | Val Loss: 0.2815 | V
al Acc: 0.9200
100%|██████████| 70/70 [00:00<00:00, 1005.28it/s]
Epoch 5/10 | Train Loss: 0.2481 | Train Acc: 0.9086 | Val Loss: 0.2672 | V
al Acc: 0.9200
100%|██████████| 70/70 [00:00<00:00, 996.06it/s]
Epoch 6/10 | Train Loss: 0.2366 | Train Acc: 0.9086 | Val Loss: 0.2557 | V
al Acc: 0.9200
100%|██████████| 70/70 [00:00<00:00, 910.68it/s]
Epoch 7/10 | Train Loss: 0.2272 | Train Acc: 0.9129 | Val Loss: 0.2461 | V
al Acc: 0.9200
100%|██████████| 70/70 [00:00<00:00, 747.63it/s]
```

```
Epoch 8/10 | Train Loss: 0.2191 | Train Acc: 0.9157 | Val Loss: 0.2377 | V
al Acc: 0.9133
100%|██████████| 70/70 [00:00<00:00, 740.61it/s]
Epoch 9/10 | Train Loss: 0.2120 | Train Acc: 0.9171 | Val Loss: 0.2302 | V
al Acc: 0.9133
100%|██████████| 70/70 [00:00<00:00, 745.40it/s]
Epoch 10/10 | Train Loss: 0.2057 | Train Acc: 0.9186 | Val Loss: 0.2235 |
Val Acc: 0.9200

Test Accuracy: 94.00%
```

Problem 1: MLP for FashionMNIST (18 points)

Now you will train a multi-layer perceptron model on the FashionMNIST dataset. Your deliverables are as follows:

1. Code for training an MLP on FashionMNIST.
2. A plot of the training and validation loss for at least 8 epochs.
3. A plot of the training and validation accuracy for each epoch, achieving a final validation accuracy of at least 82% by the end of the training.

Below we will create the training and validation datasets for you. It is on you to implement an MLP / Feed Forward neural network yourself. Please leverage the example training loop from above.

Here are some pytorch components that you should definitely use:

1. `nn.Linear`
2. Some activation: `nn.ReLU`, `nn.Tanh`, `nn.Sigmoid`, etc.
3. `nn.CrossEntropyLoss`

Here are some challenges that you will need to overcome:

1. The data is, by default, configured in image form, i.e. a (28 x 28) tensor per sample, instead of single feature vector. You will need to **reshape** it somewhere to feed it in as vector to the MLP. There are many ways of doing this according to **Problem 0**.
2. You need to write code for plotting.
3. You need to find the appropriate hyper-parameters to achieve good accuracy.

Your underlying model must be fully connected or "dense", and may not use any convolutions etc., but you can use anything in `torch.optim` or any layers in `torch.nn` besides `nn.Linear` that do not have weights.

In [14]:
```python
# Creating the datasets
# feel free to modify this as you see fit.
transform = torchvision.transforms.ToTensor()

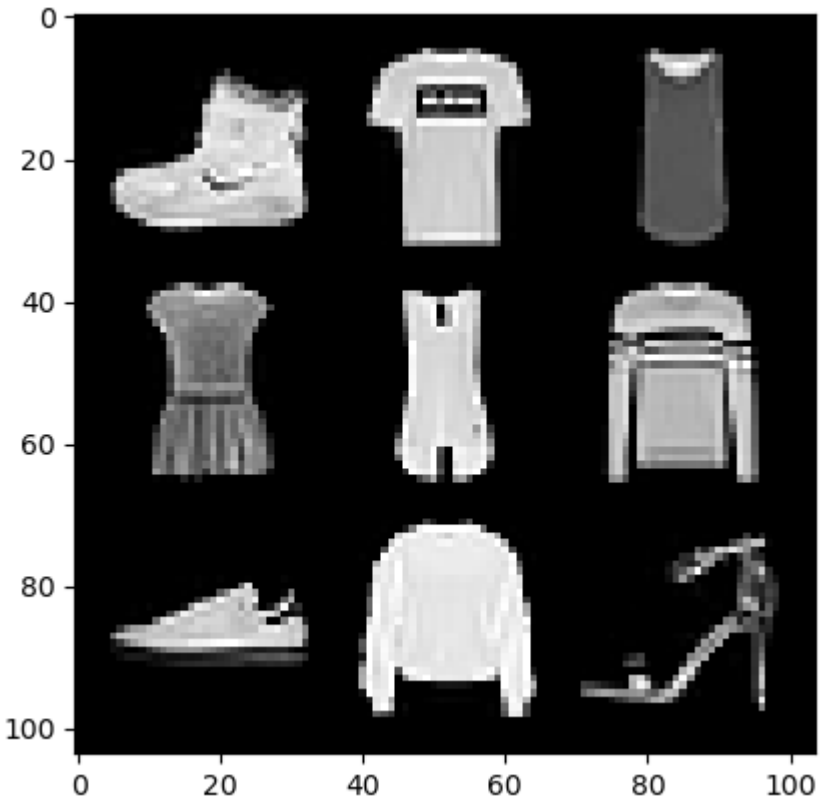
training_data = torchvision.datasets.FashionMNIST(
    root="./data",
    train=True,
    download=False,
```

```
    transform=transform,
)

validation_data = torchvision.datasets.FashionMNIST(
    root="./data",
    train=False,
    download=False,
    transform=transform,
)
```

In [15]:
```
images = [training_data[i][0] for i in range(9)]
plt.imshow(torchvision.utils.make_grid(torch.stack(images),
           nrow=3, padding=5).numpy().transpose((1, 2, 0)))
```

Out[15]: <matplotlib.image.AxesImage at 0x7fccac0eeb00>



In [16]:
```
# Get the knowledge of the Training and Validation Set
print("number of training samples: " + str(len(training_data)) + "\n" +
      "number of validation samples: " + str(len(validation_data)))
print("datatype of the 1st training sample: ", training_data[0][0].type())
print("size of the 1st training sample: ", training_data[0][0].size())

# Find out how many categories in the sample.
max_label = float("-inf")
min_label = float("inf")
for i in range(len(training_data)):
  if training_data[i][1] > max_label:
    max_label = training_data[i][1]
  if training_data[i][1] < min_label:
    min_label = training_data[i][1]
print("max_label = "+str(max_label))
print("min_label = "+str(min_label))
```

```
number of training samples: 60000
number of validation samples: 10000
datatype of the 1st training sample:  torch.FloatTensor
size of the 1st training sample:  torch.Size([1, 28, 28])
max_label = 9
min_label = 0
```

In the cell below, you'll implement a MLP. Please follow the guidence in comments.

Scoring criteria:

1. Print out the train loss, train accuracy, validation loss, and validation accuracy for each epoch. (8 points)

2. Grading criteria:

At the last epoch, if the validation accuracy (validation_acc) of your model is greater than or equal to 0.86, you will get `10 points`.

If 0.83 <= validation_acc < 0.86, you will get `6 points`.

If 0.80 <= validation_acc < 0.83, you will get `4 points`.

If validation_acc < 0.80, you will not get any score.

You are free to use the *early stopping* strategy that can prevent overfitting.

In [17]:
```
############## YOUR CODE HERE ##############
# In this part, please follow our instruction step by step to get familia

# 1. Confirm whether the CUDA is available or not.
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device", device)

# 2. TODO: Encapsulate your Multilayer Perceptron (MLP) model within a cl
# This class should include a constructor function __init__(...) and a fo
# You can modify these two functions' input parameters.
class MLP(nn.Module):
    def __init__(self,  n_inputs, n_outputs):
    ######## YOUR CODE HERE ########
        super(MLP, self).__init__()
        self.layers = nn.Sequential(
            nn.Linear(n_inputs, 256),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, n_outputs)
        )
    ######## END YOUR CODE ########

    def forward(self, x):
    ######## YOUR CODE HERE ########
        x = x.view(x.size(0), -1)  # Flatten the input
        return self.layers(x)
    ######## END YOUR CODE ########


# 3. TODO: Initialize your model with proper input size and output size
######## YOUR CODE HERE ########
n_inputs = 28 * 28
```

```
n_outputs = 10
model = MLP(n_inputs, n_outputs)
model.to(device)
####### END YOUR CODE ########

# 4. TODO: Define the Training Parameters like epochs, batch_size, learni
######## YOUR CODE HERE ########
epochs = 50
batch_size = 64
learning_rate = 0.001
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
criterion = nn.CrossEntropyLoss()
######## END YOUR CODE ########

# 5. TODO: Put the training and testing data into a DataLoader
# you can use torch.utils.data.DataLoader() to complete this step.
######## YOUR CODE HERE ########
train_loader = DataLoader(training_data, batch_size=batch_size, shuffle=T
validation_loader = DataLoader(validation_data, batch_size=batch_size, sh
######## END YOUR CODE #######

# 6. Training
# During the training process, make sure to add the training accuracy, va
# This will allow you to visualize these metrics and assess whether the m
train_acc = []
valid_acc = []
train_loss = []
valid_loss = []

model.train()  # Put model in training mode
patience = 5
best_val_loss = float("inf")
for epoch in range(epochs):
    # TODO: Please mimic the training example of logistic regression abov
    ######## YOUR CODE HERE ########
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for i, (images, labels) in enumerate(train_loader):
        # Move tensors to the configured device
        images = images.to(device)
        labels = labels.to(device)

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Track statistics
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
    # Calculate training metrics
    epoch_loss = running_loss / len(train_loader)
    epoch_acc = correct / total
    train_loss.append(epoch_loss)
    train_acc.append(epoch_acc)

    model.eval()
    val_running_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for images, labels in validation_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            val_running_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

    # Calculate validation metrics
    val_epoch_loss = val_running_loss / len(validation_loader)
    val_epoch_acc = val_correct / val_total
    valid_loss.append(val_epoch_loss)
    valid_acc.append(val_epoch_acc)

    # Print epoch results
    print(f'Epoch [{epoch+1}/{epochs}], '
          f'Train Loss: {epoch_loss:.4f}, Train Acc: {epoch_acc:.4f}, '
          f'Val Loss: {val_epoch_loss:.4f}, Val Acc: {val_epoch_acc:.4f}'


    if val_epoch_loss < best_val_loss:
        best_val_loss = val_epoch_loss
        patience_counter = 0
        # Save the best model if needed
        # torch.save(model.state_dict(), 'best_model.pth')
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f'Early stopping at epoch {epoch+1}')
            break

print("Best validation accuracy:", max(valid_acc))

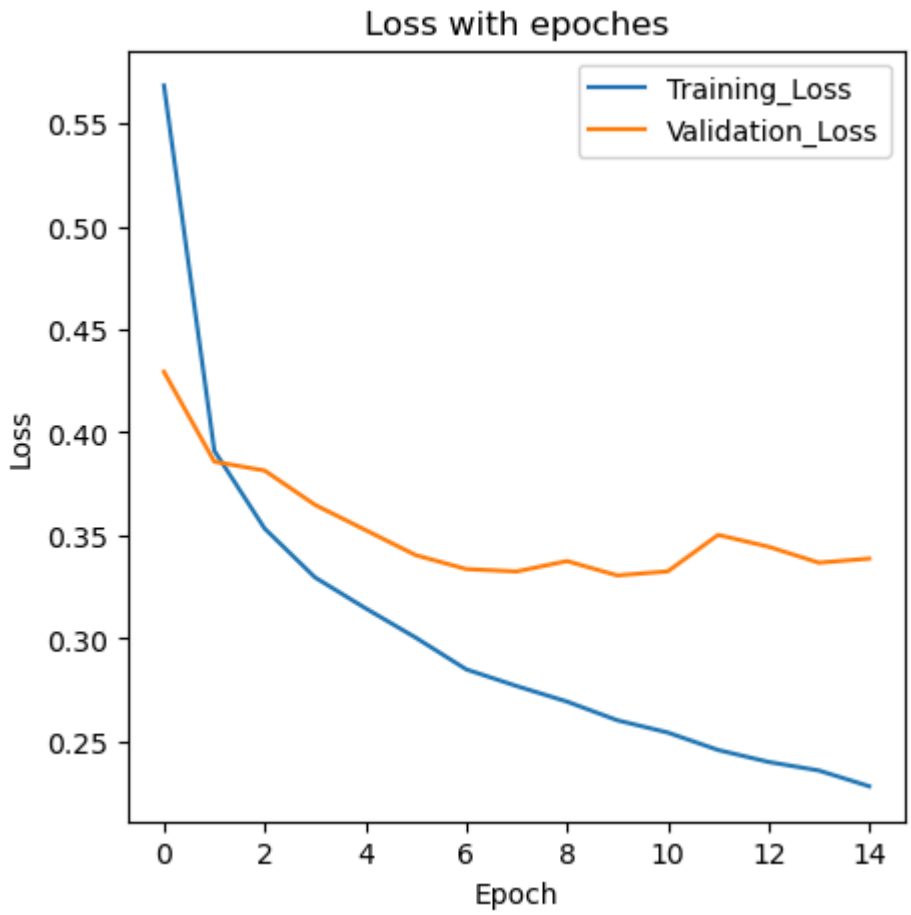
    ######## END YOUR CODE ########
```

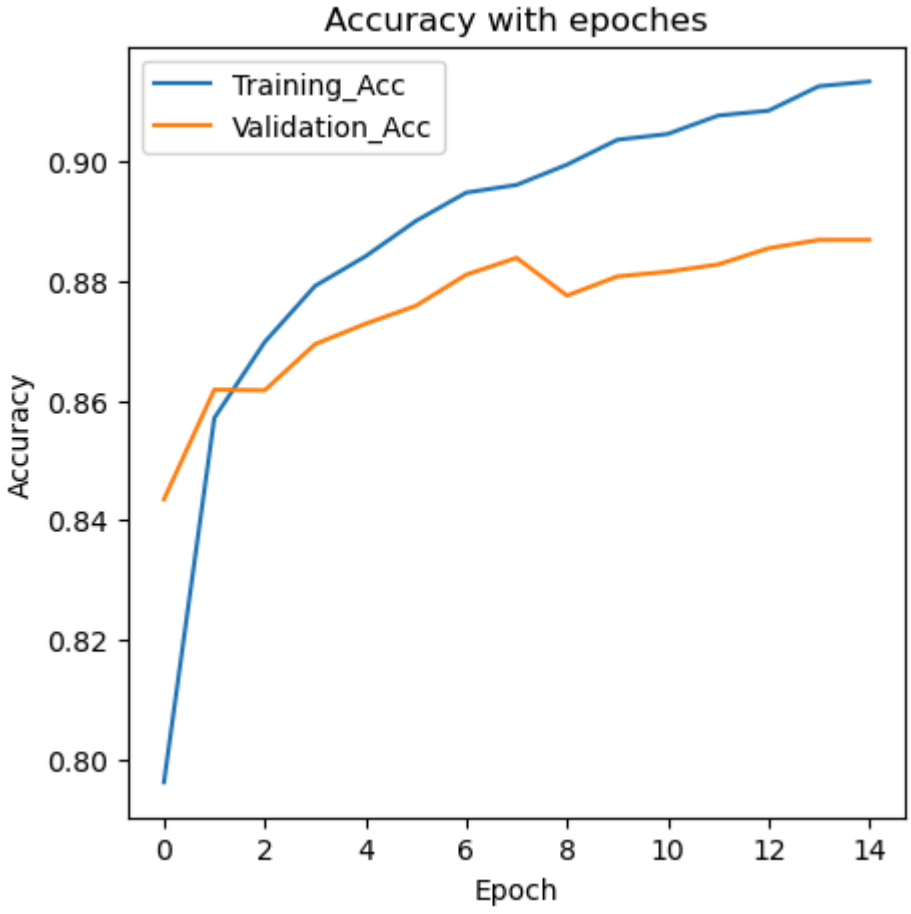
```
Using device cuda
Epoch [1/50], Train Loss: 0.5684, Train Acc: 0.7963, Val Loss: 0.4294, Val
Acc: 0.8435
Epoch [2/50], Train Loss: 0.3911, Train Acc: 0.8571, Val Loss: 0.3858, Val
Acc: 0.8618
Epoch [3/50], Train Loss: 0.3533, Train Acc: 0.8698, Val Loss: 0.3815, Val
Acc: 0.8617
Epoch [4/50], Train Loss: 0.3296, Train Acc: 0.8791, Val Loss: 0.3648, Val
Acc: 0.8694
Epoch [5/50], Train Loss: 0.3147, Train Acc: 0.8841, Val Loss: 0.3526, Val
Acc: 0.8728
Epoch [6/50], Train Loss: 0.3004, Train Acc: 0.8900, Val Loss: 0.3404, Val
Acc: 0.8758
Epoch [7/50], Train Loss: 0.2850, Train Acc: 0.8947, Val Loss: 0.3337, Val
Acc: 0.8810
Epoch [8/50], Train Loss: 0.2769, Train Acc: 0.8960, Val Loss: 0.3325, Val
Acc: 0.8838
Epoch [9/50], Train Loss: 0.2694, Train Acc: 0.8993, Val Loss: 0.3376, Val
Acc: 0.8775
Epoch [10/50], Train Loss: 0.2603, Train Acc: 0.9035, Val Loss: 0.3306, Va
l Acc: 0.8807
Epoch [11/50], Train Loss: 0.2543, Train Acc: 0.9045, Val Loss: 0.3326, Va
l Acc: 0.8815
Epoch [12/50], Train Loss: 0.2459, Train Acc: 0.9075, Val Loss: 0.3503, Va
l Acc: 0.8827
Epoch [13/50], Train Loss: 0.2401, Train Acc: 0.9083, Val Loss: 0.3445, Va
l Acc: 0.8854
Epoch [14/50], Train Loss: 0.2359, Train Acc: 0.9124, Val Loss: 0.3368, Va
l Acc: 0.8868
Epoch [15/50], Train Loss: 0.2283, Train Acc: 0.9132, Val Loss: 0.3388, Va
l Acc: 0.8868
Early stopping at epoch 15
Best validation accuracy: 0.8868
```

In [18]:

```python
# plot
plt.figure(figsize=(5, 5))
plt.plot(train_loss, label='Training_Loss')
plt.plot(valid_loss, label='Validation_Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title("Loss with epoches")
plt.legend()
plt.show()

plt.figure(figsize=(5, 5))
plt.plot(train_acc, label='Training_Acc')
plt.plot(valid_acc, label='Validation_Acc')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title("Accuracy with epoches")
plt.legend()
plt.show()
```





Problem 2: CNN for CIFAR-10 (20 points)

In this section, you will construct a Convolutional Neural Network (CNN) for the CIFAR - 10 dataset. You have already utilized this dataset in the coding part of Homework 2. However, in this particular part, there is no need to download the dataset separately. It is advisable to employ GPU acceleration for this section to enhance the operational efficiency. Nevertheless, this is not a mandatory requirement.

Here are some of the components you should consider using:

1. `nn.Conv2d`
2. `nn.ReLU`
3. `nn.Linear`
4. `nn.CrossEntropyLoss`
5. `nn.MaxPooling2d` (Optional, many implementations without it exist)

We encourage you to explore different ways of improving your model to obtain higher accuracies. Here are some suggestions for things to look into:

1. Popular CNN architectures like ResNets, etc.
2. Different optimizers and their parameters (see `torch.optim`)
3. Image preprocessing / data augmentation (see `torchvision.transforms`)
4. Regularization or dropout (see `torch.optim` and `torch.nn` respectively)
5. Learning rate scheduling: https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate

Though we encourage you to explore, there are some rules:

1. You are not allowed to use any pre-defined architectures or feature extractors in your network.
2. You are not allowed to use **any** pretrained weights, i.e. no transfer learning.
3. You cannot train on the test data (that would pretty much defeat the whole point of machine learning).

Scoring Criteria:

1. The final test accuracy of your model should be ≥ 0.80 to obtain **10 points**.
   If $0.7 \leq$ test_acc < 0.8, you will get **5 points**.
   If test_acc < 0.7, no score will be awarded.
2. Print out the train loss, train accuracy, validation loss, and validation accuracy for each epoch.**(5 points)**
3. Provide at least one training curve for your model. This curve should depict the training loss and validation loss per epoch or step after training for at least 10 epochs. **(5 points)**

In [19]:
```
# Creating the datasets, feel free to change this as long as you do the s
# You can also modify this to split the data into training and validation
# See https://pytorch.org/docs/stable/data.html#torch.utils.data.random_s

# Training and Validation transforms
transform_train = transforms.Compose([
```

```
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.201
])

# Testing transform
transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.201
])

# Training and validation data
train_val_data = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=False,
    transform=transform_train,
)

train_size = int(0.9*len(train_val_data))
val_size = len(train_val_data) - train_size

# 9:1 randomly split the original training set to training and validation
generator = torch.Generator().manual_seed(42)

train_data, valid_data = torch.utils.data.random_split(
    train_val_data,
    [train_size, val_size],
    generator=generator
)

# Official testing set.
test_data = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=False,
    transform=transform_test,
)

print(f"Training size: {len(train_data)}")
print(f"Validation size: {len(valid_data)}")
print(f"Testing size: {len(test_data)}")
```

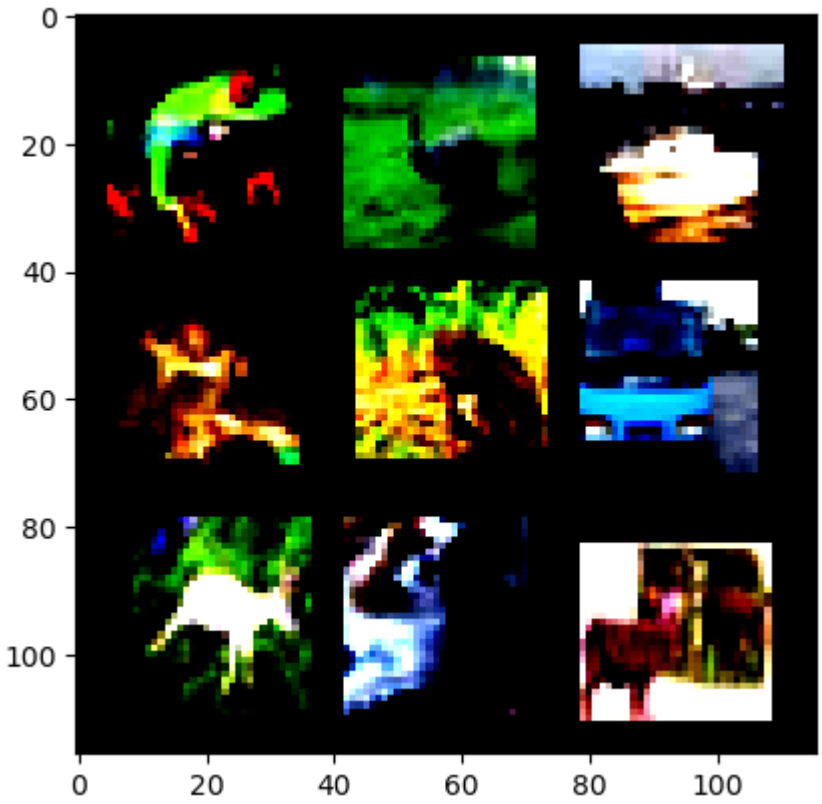
```
Training size: 45000
Validation size: 5000
Testing size: 10000
```

Again, let's first visualize our data.

In [20]:
```
images = [train_data[i][0] for i in range(9)]
print(images[0].size())
plt.imshow(torchvision.utils.make_grid(torch.stack(images),
           nrow=3, padding=5).numpy().transpose((1, 2, 0)))
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers). Got range [-2.4290657..2.7537313].
torch.Size([3, 32, 32])
```

Out[20]: <matplotlib.image.AxesImage at 0x7fcc9c25f2b0>



In [21]:
```
print("number of training samples: " + str(len(train_data)) + "\n" +
      "number of testing samples: " + str(len(valid_data)))
print("datatype of the 1st training sample: ", train_data[0][0].type())
print("size of the 1st training sample: ", train_data[0][0].size())
```

```
number of training samples: 45000
number of testing samples: 5000
datatype of the 1st training sample:  torch.FloatTensor
size of the 1st training sample:  torch.Size([3, 32, 32])
```

In [22]:
```
max_label = float("-inf")
min_label = float("inf")
for i in range(len(train_data)):
  if train_data[i][1] > max_label:
    max_label = train_data[i][1]
  if train_data[i][1] < min_label:
    min_label = train_data[i][1]
print("max_label = "+str(max_label))
print("min_label = "+str(min_label))
```

```
max_label = 9
min_label = 0
```

CNN Construction and Training

Here are some training tips for CNN training.

1. **Optimization for Gradient Descent.**
   A. **Stochastic gradient descent (SGD)**
   `torch.optim.SGD`
   When updating model parameters using only one training sample at each epoch, compared to traditional gradient descent methods that process the

entire dataset, the memory and computational requirements are lower, which can accelerate convergence. Additionally, the randomness in the optimization process helps escape local minima. However, the noise in the updates may lead to a more erratic optimization path, unstable convergence speed, and the choice of learning rate becomes particularly crucial. Moreover, you can use SGD with momentum. The momentum parameter (typically between 0.9 and 0.99) controls the influence of past gradients on the current update, enabling the algorithm to persistently move in promising directions.

B. **RMSprop**
`torch.optim.RMSprop`
This method adaptively adjusts the learning rate based on the recent gradient history of parameters: it reduces the learning rate for parameters with consistently large gradients while increasing the update magnitude for those with smaller gradients, thereby balancing the step sizes of different parameter updates. The `alpha` parameter (decay rate, typically set to 0.9) in RMSprop controls how quickly old squared gradients are forgotten.

C. **Adam**
`torch.optim.Adam`
Adam optimizer efficiently updates parameters by combining momentum (1st-order moment) and RMSprop (2nd-order moment) with adaptive learning rates and bias correction.

2. **Balancing hyperparameters (epochs, batch size, learning rate) is crucial during training.**
3. **Dealing with Overfitting**:
   A. **Batch Normalization**:
   `torch.nn.BatchNorm2d`
   Batch normalization mitigates internal covariate shift by normalizing and rescaling layer inputs within each mini-batch.
   B. **MaxPooling Layers**:
   `nn.MaxPool2d`
   spatial downsampling
   C. **Dropout**:
   `torch.nn.Dropout`
   Dropout randomly deactivates neurons during training to prevent overfitting and improve generalization.
   D. **L1/L2 Normalization**:
   Configure the `weight_decay` parameter in the `optimizer`.
   E. **Early Stopping**
   Implement early stopping by continuously monitoring validation performance and halting training when no improvement is observed for a predefined patience period.
4. **Dealing with Underfitting**:
   A. **Deeper networks**
   B. **More training epochs**

C. **Reasonably select activate functions**

D. **Use a classic network architecture, e.g. ResNet.**

5. You are encouraged to add more optimization in order to reach your goal.

In [23]:

```
# 1. Confirm whether the CUDA is available or not.
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device", device)

# 2. TODO: Encapsulate your Convolutional Neural Network (CNN) model with
# This class should include a constructor function __init__(...) and a fo
class CNN(nn.Module):
    def __init__(self):
        ######## YOUR CODE HERE ########
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)

        self.fc1 = nn.Linear(128 * 4 * 4, 512)  # Adjust input size based
        self.fc2 = nn.Linear(512, 10)  # 10 classes for CIFAR-10

        self.dropout = nn.Dropout(0.1)
        ######## END YOUR CODE ########

    def forward(self, x):
        ######## YOUR CODE HERE ########
        x = self.pool(F.relu(self.conv1(x)))

        x = self.pool(F.relu(self.conv2(x)))

        x = self.pool(F.relu(self.conv3(x)))

        x = x.view(-1, 128 * 4 * 4)  # Flatten the tensor

        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)

        return x
        ######## END YOUR CODE ########


# 3. Initialize your model
model = CNN().to(device)

# 4. TODO: Define the Training Parameters like epochs, batch_size, learni
######## YOUR CODE HERE ########
epochs = 50
batch_size = 64
learning_rate = 0.001
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
criterion = nn.CrossEntropyLoss()
######## END YOUR CODE ########

# 5. TODO: Put the training and testing data into a DataLoader
# you can use torch.utils.data.DataLoader() to complete this step.
######## YOUR CODE HERE ########
train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True
```

```
validation_loader = DataLoader(valid_data, batch_size=batch_size, shuffle
######## END YOUR CODE ########

# 6. Training
# During the training process, make sure to add the training accuracy, va
# This will allow you to visualize these metrics and assess whether the m
train_acc = []
valid_acc = []
train_loss = []
valid_loss = []

model.train()  # Put model in training mode
patience = 10
best_val_loss = float("inf")
for epoch in range(epochs):
    # TODO: Please mimic the training example of logistic regression abov
    ######## YOUR CODE HERE ########
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for i, (images, labels) in enumerate(train_loader):
        # Move tensors to the configured device
        images = images.to(device)
        labels = labels.to(device)

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Track statistics
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    # Calculate training metrics
    epoch_loss = running_loss / len(train_loader)
    epoch_acc = correct / total
    train_loss.append(epoch_loss)
    train_acc.append(epoch_acc)

    # Validation phase

    model.eval()
    val_running_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for images, labels in validation_loader:
            images = images.to(device)
            labels = labels.to(device)
```

```
            outputs = model(images)
            loss = criterion(outputs, labels)

            val_running_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

    # Calculate validation metrics
    val_epoch_loss = val_running_loss / len(validation_loader)
    val_epoch_acc = val_correct / val_total
    valid_loss.append(val_epoch_loss)
    valid_acc.append(val_epoch_acc)

    # Print epoch results
    print(f'Epoch [{epoch+1}/{epochs}], '
          f'Train Loss: {epoch_loss:.4f}, Train Acc: {epoch_acc:.4f}, '
          f'Val Loss: {val_epoch_loss:.4f}, Val Acc: {val_epoch_acc:.4f}')

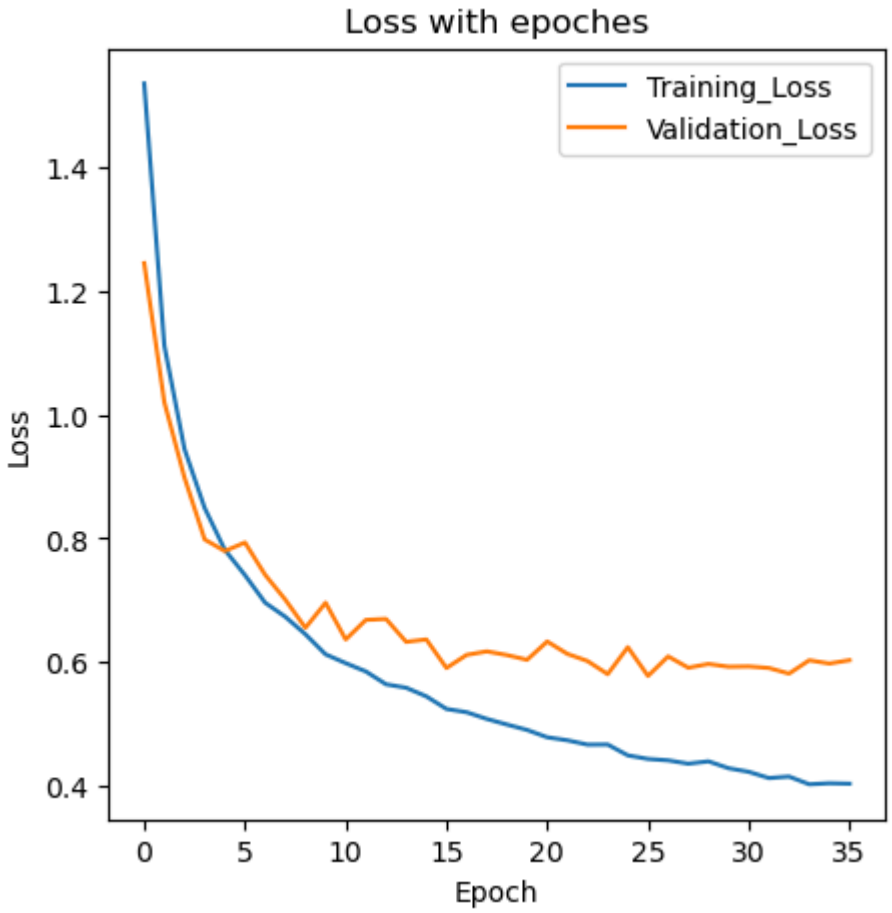
    # Early stopping check
    if val_epoch_loss < best_val_loss:
        best_val_loss = val_epoch_loss
        patience_counter = 0
        # Save the best model if needed
        # torch.save(model.state_dict(), 'best_model.pth')
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f'Early stopping at epoch {epoch+1}')
            break

    ######## END YOUR CODE ########
```

```
Using device cuda
Epoch [1/50], Train Loss: 1.5339, Train Acc: 0.4362, Val Loss: 1.2440, Val
Acc: 0.5522
Epoch [2/50], Train Loss: 1.1110, Train Acc: 0.6028, Val Loss: 1.0197, Val
Acc: 0.6270
Epoch [3/50], Train Loss: 0.9441, Train Acc: 0.6676, Val Loss: 0.8981, Val
Acc: 0.6816
Epoch [4/50], Train Loss: 0.8492, Train Acc: 0.7002, Val Loss: 0.7983, Val
Acc: 0.7182
Epoch [5/50], Train Loss: 0.7816, Train Acc: 0.7258, Val Loss: 0.7796, Val
Acc: 0.7210
Epoch [6/50], Train Loss: 0.7408, Train Acc: 0.7412, Val Loss: 0.7934, Val
Acc: 0.7242
Epoch [7/50], Train Loss: 0.6963, Train Acc: 0.7558, Val Loss: 0.7413, Val
Acc: 0.7364
Epoch [8/50], Train Loss: 0.6736, Train Acc: 0.7649, Val Loss: 0.7013, Val
Acc: 0.7550
Epoch [9/50], Train Loss: 0.6454, Train Acc: 0.7746, Val Loss: 0.6558, Val
Acc: 0.7702
Epoch [10/50], Train Loss: 0.6130, Train Acc: 0.7877, Val Loss: 0.6962, Va
l Acc: 0.7596
Epoch [11/50], Train Loss: 0.5987, Train Acc: 0.7897, Val Loss: 0.6370, Va
l Acc: 0.7800
Epoch [12/50], Train Loss: 0.5853, Train Acc: 0.7962, Val Loss: 0.6686, Va
l Acc: 0.7718
Epoch [13/50], Train Loss: 0.5646, Train Acc: 0.8026, Val Loss: 0.6701, Va
l Acc: 0.7696
Epoch [14/50], Train Loss: 0.5591, Train Acc: 0.8054, Val Loss: 0.6331, Va
l Acc: 0.7840
Epoch [15/50], Train Loss: 0.5452, Train Acc: 0.8077, Val Loss: 0.6370, Va
l Acc: 0.7812
Epoch [16/50], Train Loss: 0.5249, Train Acc: 0.8158, Val Loss: 0.5910, Va
l Acc: 0.7896
Epoch [17/50], Train Loss: 0.5196, Train Acc: 0.8189, Val Loss: 0.6121, Va
l Acc: 0.7892
Epoch [18/50], Train Loss: 0.5088, Train Acc: 0.8210, Val Loss: 0.6177, Va
l Acc: 0.7846
Epoch [19/50], Train Loss: 0.4997, Train Acc: 0.8241, Val Loss: 0.6118, Va
l Acc: 0.7882
Epoch [20/50], Train Loss: 0.4910, Train Acc: 0.8277, Val Loss: 0.6042, Va
l Acc: 0.7962
Epoch [21/50], Train Loss: 0.4791, Train Acc: 0.8343, Val Loss: 0.6336, Va
l Acc: 0.7944
Epoch [22/50], Train Loss: 0.4743, Train Acc: 0.8337, Val Loss: 0.6137, Va
l Acc: 0.7886
Epoch [23/50], Train Loss: 0.4673, Train Acc: 0.8368, Val Loss: 0.6021, Va
l Acc: 0.8030
Epoch [24/50], Train Loss: 0.4674, Train Acc: 0.8383, Val Loss: 0.5810, Va
l Acc: 0.8080
Epoch [25/50], Train Loss: 0.4499, Train Acc: 0.8423, Val Loss: 0.6245, Va
l Acc: 0.7900
Epoch [26/50], Train Loss: 0.4442, Train Acc: 0.8459, Val Loss: 0.5781, Va
l Acc: 0.8026
Epoch [27/50], Train Loss: 0.4419, Train Acc: 0.8445, Val Loss: 0.6097, Va
l Acc: 0.8010
Epoch [28/50], Train Loss: 0.4365, Train Acc: 0.8484, Val Loss: 0.5914, Va
l Acc: 0.8050
Epoch [29/50], Train Loss: 0.4403, Train Acc: 0.8454, Val Loss: 0.5975, Va
l Acc: 0.7984
Epoch [30/50], Train Loss: 0.4292, Train Acc: 0.8481, Val Loss: 0.5930, Va
```

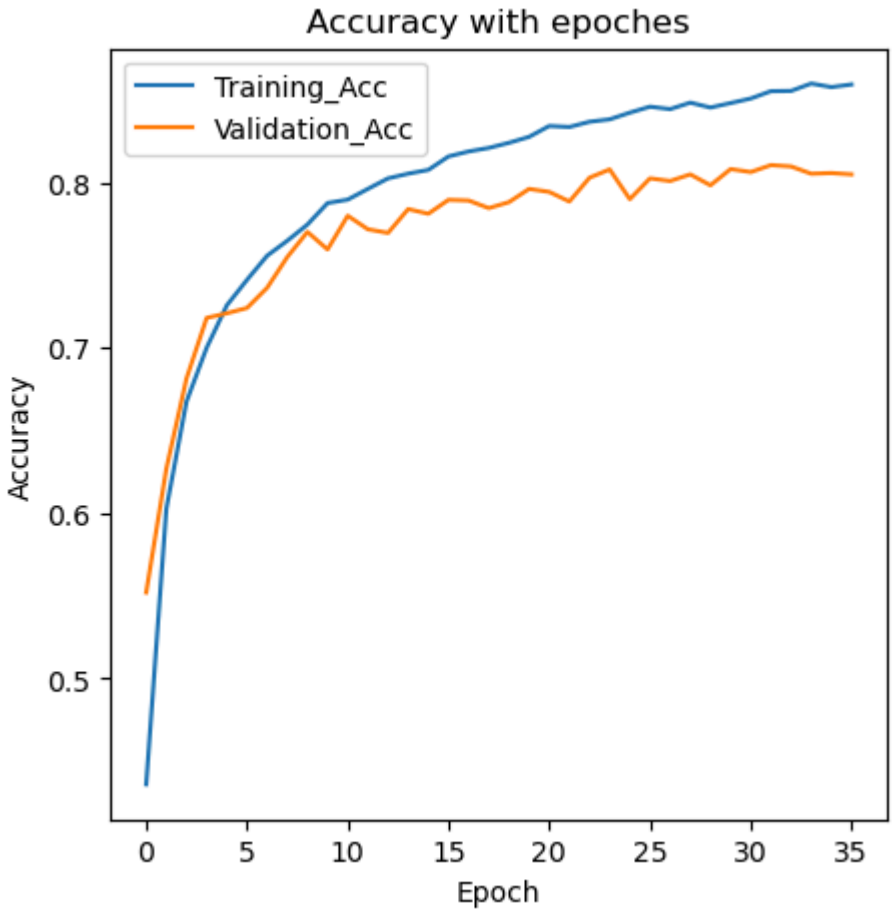
```
l Acc: 0.8082
Epoch [31/50], Train Loss: 0.4234, Train Acc: 0.8509, Val Loss: 0.5934, Va
l Acc: 0.8064
Epoch [32/50], Train Loss: 0.4134, Train Acc: 0.8553, Val Loss: 0.5910, Va
l Acc: 0.8106
Epoch [33/50], Train Loss: 0.4157, Train Acc: 0.8555, Val Loss: 0.5819, Va
l Acc: 0.8098
Epoch [34/50], Train Loss: 0.4033, Train Acc: 0.8599, Val Loss: 0.6034, Va
l Acc: 0.8054
Epoch [35/50], Train Loss: 0.4050, Train Acc: 0.8578, Val Loss: 0.5981, Va
l Acc: 0.8058
Epoch [36/50], Train Loss: 0.4043, Train Acc: 0.8594, Val Loss: 0.6035, Va
l Acc: 0.8050
Early stopping at epoch 36
```

In [24]:
```python
# Plot Your Training and Validation loss in one picture here.
plt.figure(figsize=(5, 5))
plt.plot(train_loss, label='Training_Loss')
plt.plot(valid_loss, label='Validation_Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title("Loss with epoches")
plt.legend()
plt.show()
```



In [25]:
```python
# Plot Your Training and Validation accuracy in one picture here.
plt.figure(figsize=(5, 5))
plt.plot(train_acc, label='Training_Acc')
plt.plot(valid_acc, label='Validation_Acc')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title("Accuracy with epoches")
```

```
plt.legend()
plt.show()
```



Testing

In [26]:
```
def evaluate_model(model, test_data, device='cuda', batch_size=128):
    test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=Fa
    model.eval()
    model.to(device)
    correct = 0
    total = 0
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
    accuracy = 100 * correct / total

    print(f'Test Accuracy: {accuracy:.2f}%')
    print(f'Correct/Total: {correct}/{total}')
    print('\nClassification Report:')
    print(classification_report(all_labels, all_preds, target_names=test_
```

In [27]:
```
evaluate_model(model, test_data)
```

```
Test Accuracy: 82.51%
Correct/Total: 8251/10000

Classification Report:
              precision    recall  f1-score   support

    airplane       0.85      0.82      0.83      1000
  automobile       0.94      0.89      0.92      1000
        bird       0.76      0.77      0.76      1000
         cat       0.71      0.65      0.68      1000
        deer       0.81      0.81      0.81      1000
         dog       0.77      0.76      0.76      1000
        frog       0.82      0.90      0.85      1000
       horse       0.84      0.87      0.85      1000
        ship       0.86      0.91      0.89      1000
       truck       0.89      0.88      0.88      1000

    accuracy                           0.83     10000
   macro avg       0.82      0.83      0.82     10000
weighted avg       0.82      0.83      0.82     10000
```